

EXPERIMENT-3

C Programming – Debugging Tools

1) Fundamentals of Errors and Debugging:

Definitions:

- **Error** – A mistake or blunder made by the programmer in the source code.
Example – Writing a/b when the intention was $a + b$.
- **Bug** – Bug is a flaw in a computer program which causes the program to produce a wrong or unexpected result.
Example – The program didn't print the output it was supposed to or it shows the product instead of sum.
- **Debugging** – It is the process of finding, analyzing and fixing bugs in a computer program.

2) Classification of Errors in C:

Error Type	Description	Primary Cause	Example
Syntax Errors	Code violates the grammatical rules of the C language.	Missing semicolons, unmatched braces, typos in keywords.	<code>printf("Hello")</code>
Compilation/Type errors	Operations performed on incompatible types or missing declarations.	Assigning float string to int pointer, using undeclared variable.	<code>int a = "Hello";</code>
Linker Errors	The compiler generates object files, but the linker cannot combine them.	Function declared but not defined, missing libraries.	undefined reference to 'main'
Runtime Errors	The program crashes or terminates abnormally during execution.	Division by zero, accessing invalid memory addresses.	<code>int var1 = 20/ 0;</code>
Logical errors	Code runs to completion without crashing, but output is incorrect	Wrong mathematical formula, incorrect loop condition.	<code>for(i=0; i<5; i++)</code> instead of <code><=5</code>

Why Does a Program Compile Successfully but Produce Incorrect Output?

A compiler's job is to verify that your code obeys the syntax and type rules of the C language, and to convert that code into machine instructions. The compiler cannot understand human intent.

- If you write `sum = a - b`; instead of `sum = a + b`; the statement is completely valid C syntax. Therefore, compilation succeeds, but the resulting logic produces an erroneous output at execution time.

Case 1 : Syntax & Compilation Errors

When the compiler encounters invalid code, it prints diagnostics specifying the file name, line number, column, and error description.

Example Code (Experiment1.c)

```
#include <stdio.h>
int main()
{
    int var1 = 8;
    printf("%d\n", var1);
    return 0;
}
```

Compilation Command & Output

```
(kali㉿kali)-[~/Desktop]
$ vi experiment1.c

(kali㉿kali)-[~/Desktop]
$ cc experiment1.c
experiment1.c: In function 'main':
experiment1.c:5:9: error: expected ',', or ';' before 'printf'
   5 |         printf("%d\n", var1);
     |         ^~~~~~

(kali㉿kali)-[~/Desktop]
$
```

Breakdown of Error Message

- Filename: sample1.c
- Line & Column: Line 5, Column 5
- Type of Message: error
- Description: Expected , or ; before printf statement.

Case 2 : Compiler Warning (-Wall, -Wextra)

A warning indicates code that is syntactically legal, but potentially dangerous or suspicious. Warnings do not stop compilation, but ignoring them frequently leads to bugs at runtime.

Example code (Experiment1a.c)

```
#include <stdio.h>
int main()
{
    int var2;
    printf("Experiment\n");
    return 0;
}
```

#Standard Compilation vs. Warning Flags

```
(kali㉿kali)-[~/Desktop]
$ vi experiment1a.c

(kali㉿kali)-[~/Desktop]
$ cc experiment1a.c

(kali㉿kali)-[~/Desktop]
$ ./a.out
Experiment
```

Enabling comprehensive warning flags

```
(kali㉿kali)-[~/Desktop]
$ cc -Wall -Wextra -g experiment1a.c
experiment1a.c: In function 'main':
experiment1a.c:4:13: warning: unused variable 'var2' [-Wunused-variable]
   4 |         int var2;
     |         ^~~~

(kali㉿kali)-[~/Desktop]
$
```

- -Wall: Enables a broad set of commonly useful compiler warnings.
- -Wextra: Enables additional warnings not covered by -Wall.

Case-3: printf-Based Debugging (Logical bug)

Before using complex debugging tools, simple program flow and state inspection can be performed using output statements (printf).

Basic Value Tracing

```
printf("DEBUG: a = %d, b = %d\n", a, b);
```

Example: Tracking Loop Execution (experiment3.c)

The following program adds numbers from 1 to 4 instead of 1 to 5 due to a logical bug. Debug printf() statements help expose the exact iteration behavior.

```
1 #include <stdio.h>
2 int main(void)
3 {
4     int i, sum = 0;
5     for(i = 0; i < 5; i++)
6     {
7         printf("DEBUG: entering iteration i = %d\n", i);
8         sum = sum + i;
9         printf("DEBUG: sum updated to = %d\n", sum);
10    }
11    printf("Final Sum = %d\n", sum);    return 0;
12 }
```

3. Introduction to GDB (GNU Debugger)

Background: What is GNU & GDB?

- GNU: A recursive acronym for "GNU's Not Unix!". It is an open-source operating system project initiated by Richard Stallman in 1983 to provide a completely free Unix-like system.
- GDB (GNU Debugger): The standard debugger for the GNU software system, allowing developers to inspect variable values, set execution breakpoints, and analyze crashes.

Installation & Compilation for GDB

```
Session Actions Edit View Help
kali@kali: ~/Desktop

(kali@kali)-[~/Desktop]
$ sudo apt update
Hit:1 http://http.kali.org/kali kali-rolling InRelease
1133 packages can be upgraded. Run 'apt list --upgradable' to see them.

(kali@kali)-[~/Desktop]
$ sudo apt install gdb
gdb is already the newest version (17.2-1+b1).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1133

(kali@kali)-[~/Desktop]
$
```

Since gdb is already installed in my linux it is showing like this but if it's not installed you'll get a option for yes or no by simply typing 'y' for yes the installation will start.

Compile source code with debugging symbols enabled (-g)

```
(kali@kali)-[~/Desktop]
$ cc -g -Wall -Wextra experiment3.c

(kali@kali)-[~/Desktop]
$ ./a.out
DEBUG: entering iteration i = 0
DEBUG: sum updated to = 0
DEBUG: entering iteration i = 1
DEBUG: sum updated to = 1
DEBUG: entering iteration i = 2
DEBUG: sum updated to = 3
DEBUG: entering iteration i = 3
DEBUG: sum updated to = 6
DEBUG: entering iteration i = 4
DEBUG: sum updated to = 10
Final Sum = 10

(kali@kali)-[~/Desktop]
$
```

Quick Reference: Common GDB Commands

Command	Shorthand	Description/Purpose
Break<line/func>	b	Sets a breakpoint at a specified line number or function name.
Run	r	Starts execution of the program from the beginning.
Next	n	Executes the next line of code (steps over function calls).
Step	s	Executes the next line of code (steps into function calls).
Print<var>	p	Prints the current value of a variable or expression.
Display<var>	disp	Automatically prints the value of a variable after every step.
Continue	c	Resumes execution until the next breakpoint or program completion.
List	l	Prints source code lines around the current execution position.
Info locals	Info loc	Lists all local variables and their current values in the current frame.
Backtrace	bt	Displays the current call stack (chain of function calls).
Info breakpoints	l b	Displays all active breakpoints and their numbers.
Delete<id>	d	Deletes the specified breakpoint number (or all if omitted).
Disable <id>	dis	Temporarily disables a breakpoint without deleting it.
Enable<id>	en	Re-enables a previously disabled breakpoint.
Quit	q	Exits the GDB debugging session.

Case 4: Debugging a Logical Error using GDB

Example: Code with Logic Bug (experiment4.c)

```
1 #include <stdio.h>
2 int main()
3 {
4     int i , sum = 0;
5     for(i=0 ; i<5 ; i++)
6     {
7         sum = sum + i;
8     }
9     printf("sum = %d\\n",sum);
10    return 0;
11 }
```

- Expected Output: 15 (1+2+3+4+5)
- Actual Output: 10 (1+2+3+4)

Debugging Walkthrough in GDB

```
Session Actions Edit View Help
(kali@kali)-[~/Desktop]
$ cc -g experiment4.c -o experiment4
(kali@kali)-[~/Desktop]
$ gdb ./experiment4
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word" ...
Reading symbols from ./experiment4 ...
(gdb) list
1      #include <stdio.h>
2      int main()
3      {
4          int i, sum = 0;
5          for(i = 1; i<5; i++)
6          {
7              sum = sum + i;
8          }
9          printf("sum = %d\n",sum);
10         return 0;
(gdb) break 7
Breakpoint 1 at 0x1151: file experiment4.c, line 7.
(gdb) run
Starting program: /home/kali/Desktop/experiment4
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1"
.
Breakpoint 1, main () at experiment4.c:7
7          sum = sum + i;
(gdb) print i
$1 = 1
(gdb) display i
1: i = 1
(gdb) display sum
2: sum = 0
(gdb) continue
Continuing.
Breakpoint 1, main () at experiment4.c:7
7          sum = sum + i;
1: i = 2
2: sum = 1
(gdb) continue
Continuing.
Breakpoint 1, main () at experiment4.c:7
7          sum = sum + i;
1: i = 3
2: sum = 3
(gdb) continue
Continuing.
Breakpoint 1, main () at experiment4.c:7
7          sum = sum + i;
1: i = 4
2: sum = 6
(gdb) continue
Continuing.
sum = 10
[Inferior 1 (process 18790) exited normally]
(gdb)
```

Conclusion: The loop terminates when i reaches 5 without executing the loop body for 5. The loop condition should be changed from $i < 5$ to $i \leq 5$.

Case 5: Debugging Functions (next vs. step)

- **next:** Executes the line. If the line contains a function call, the function executes entirely in the background without entering its body.
- **step:** Steps into the function call, transferring GDB's focus into the function body for line-by-line inspection.

Example Code (experiment5.c)

```
1 #include <stdio.h>
2 int square(int x)
3 {
4     int result1;
5     result1 = x*x;
6     return result1;
7 }
8 int main()
9 {
10    int result, n = 5;
11    result = square(n);
12    printf("Square = %d\n", result);
13    return 0;
14 }
```

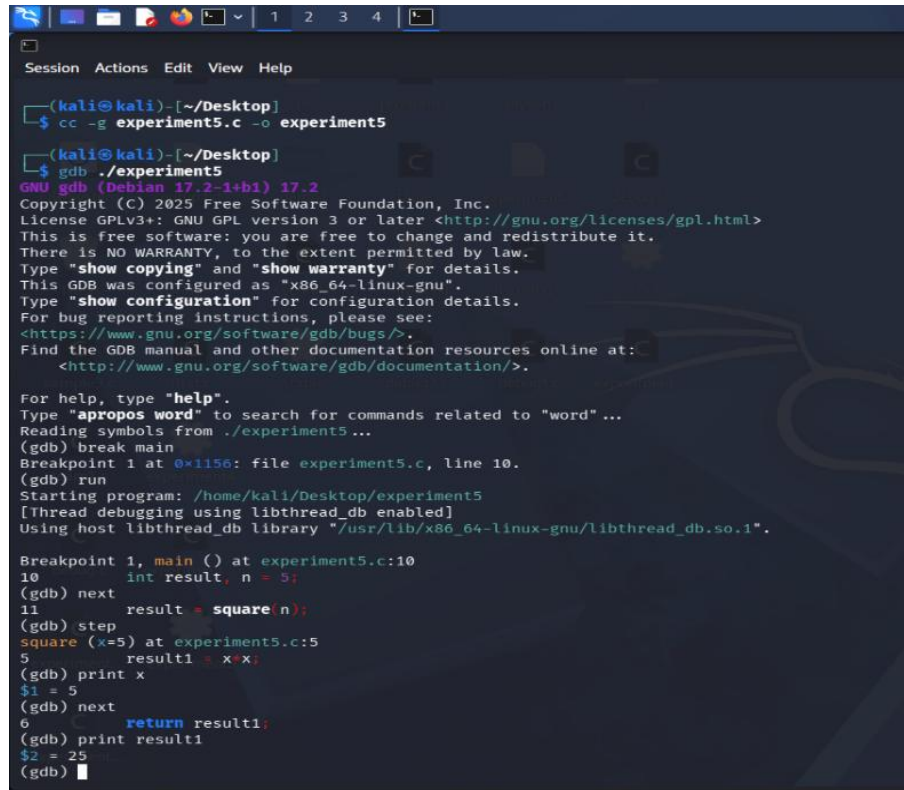
Option A : Stepping over with next

```
Session Actions Edit View Help
(kali@kali)-[~/Desktop]
$ cc -g experiment5.c -o experiment5
(kali@kali)-[~/Desktop]
$ gdb ./experiment5
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word" ...
Reading symbols from ./experiment5 ...
(gdb) break main
Breakpoint 1 at 0x1156: file experiment5.c, line 10.
(gdb) run
Starting program: /home/kali/Desktop/experiment5
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at experiment5.c:10
10    int result, n = 5;
(gdb) next
11    result = square(n);
(gdb) next
12    printf("Square = %d\n", result);
(gdb) print result
$1 = 25
(gdb)
```


Option B: Stepping into with step



```
(kali@kali)-[~/Desktop]
$ cc -g experiment5.c -o experiment5

(kali@kali)-[~/Desktop]
$ gdb ./experiment5
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

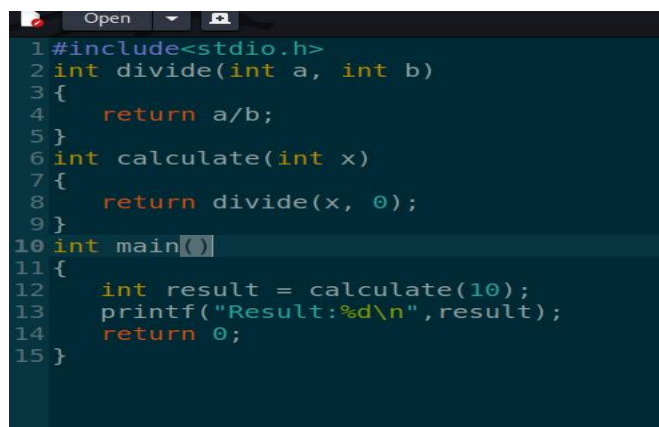
For help, type "help".
Type "apropos word" to search for commands related to "word" ...
Reading symbols from ./experiment5...
(gdb) break main
Breakpoint 1 at 0x1156: file experiment5.c, line 10.
(gdb) run
Starting program: /home/kali/Desktop/experiment5
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at experiment5.c:10
10   int result, n = 5;
(gdb) next
11   result = square(n);
(gdb) step
square (x=5) at experiment5.c:5
5   result1 = x*x;
(gdb) print x
$1 = 5
(gdb) next
6   return result1;
(gdb) print result1
$2 = 25
(gdb)
```

Case 6: Debugging Runtime Errors using Backtrace (backtrace / bt)

When a program crashes (e.g., floating point exception / division by zero), GDB can pinpoint the exact function call sequence leading to the crash.

Example Code (experiment6.c)



```
1 #include<stdio.h>
2 int divide(int a, int b)
3 {
4     return a/b;
5 }
6 int calculate(int x)
7 {
8     return divide(x, 0);
9 }
10 int main()
11 {
12     int result = calculate(10);
13     printf("Result:%d\n", result);
14     return 0;
15 }
```

GDB Backtrace Execution

```
Session Actions Edit View Help
(kali@kali) ~/Desktop
$ cc -g experiment6.c -o experiment6
(kali@kali) ~/Desktop
$ gdb ./experiment6
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (c) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word" ...
Reading symbols from ./experiment6...
(gdb) run
Starting program: /home/kali/Desktop/experiment6
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Program received signal SIGFPE, Arithmetic exception.
0x0000555555555147 in divide (a=10, b=0) at experiment6.c:4
4      return a/b;
(gdb) backtrace
#0  0x0000555555555147 in divide (a=10, b=0) at experiment6.c:4
#1  0x0000555555555166 in calculate (x=10) at experiment6.c:8
#2  0x000055555555517a in main () at experiment6.c:12
(gdb) █
```

Understanding Stack Frames

- Frame #0: The function where the crash actually occurred (divide).
- Frame #1: The function that called frame #0 (calculate).
- Frame #2: The top-level function that started the call sequence (main).

```
(gdb) frame 0
#0  0x0000555555555147 in divide (a=10, b=0) at experiment6.c:4
4      return a/b;
(gdb) print b
$1 = 0
(gdb) frame 1
#1  0x0000555555555166 in calculate (x=10) at experiment6.c:8
8      return divide(x, 0);
(gdb) print x
$2 = 10
(gdb) █
```

Case 7: Debugging Array Out-of-Bounds Errors using GDB

In C, array bounds are not checked at runtime automatically. Accessing indices beyond the declared limit results in undefined behavior or memory corruption.

Example Code (sample7.c)

```
1#include<stdio.h>
2int main()
3{
4    int a[5];
5    int i;
6    for(i = 0; i<= 5; i++)
7    {
8        scanf("%d",&a[i]);
9    }
10   return 0;
11}
```

Inspection in GDB

```
Session Actions Edit View Help
kali@kali: ~/Desktop
(kali@kali) - [~/Desktop]
$ gdb ./experiment7
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word" ...
Reading symbols from ./experiment7 ...
(gdb) break 8
Breakpoint 1 at 0x114a: file experiment7.c, line 8.
(gdb) run
Starting program: /home/kali/Desktop/experiment7
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

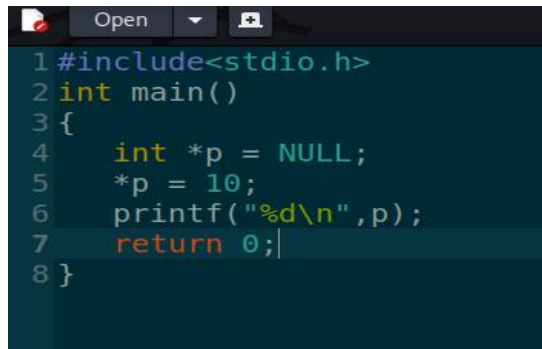
Breakpoint 1, main () at experiment7.c:8
8         scanf("%d",&a[i]);
(gdb) print &a[0]
$1 = (int *) 0x7fffffffdd10
(gdb) print &a[4]
$2 = (int *) 0x7fffffffdd20
(gdb) print &a[6]
$3 = (int *) 0x7fffffffdd28
(gdb) =
```

- Correct Loop Condition: `for(i = 0; i < 5; i++)`

Case 8: Segmentation Faults (SIGSEGV) using GDB

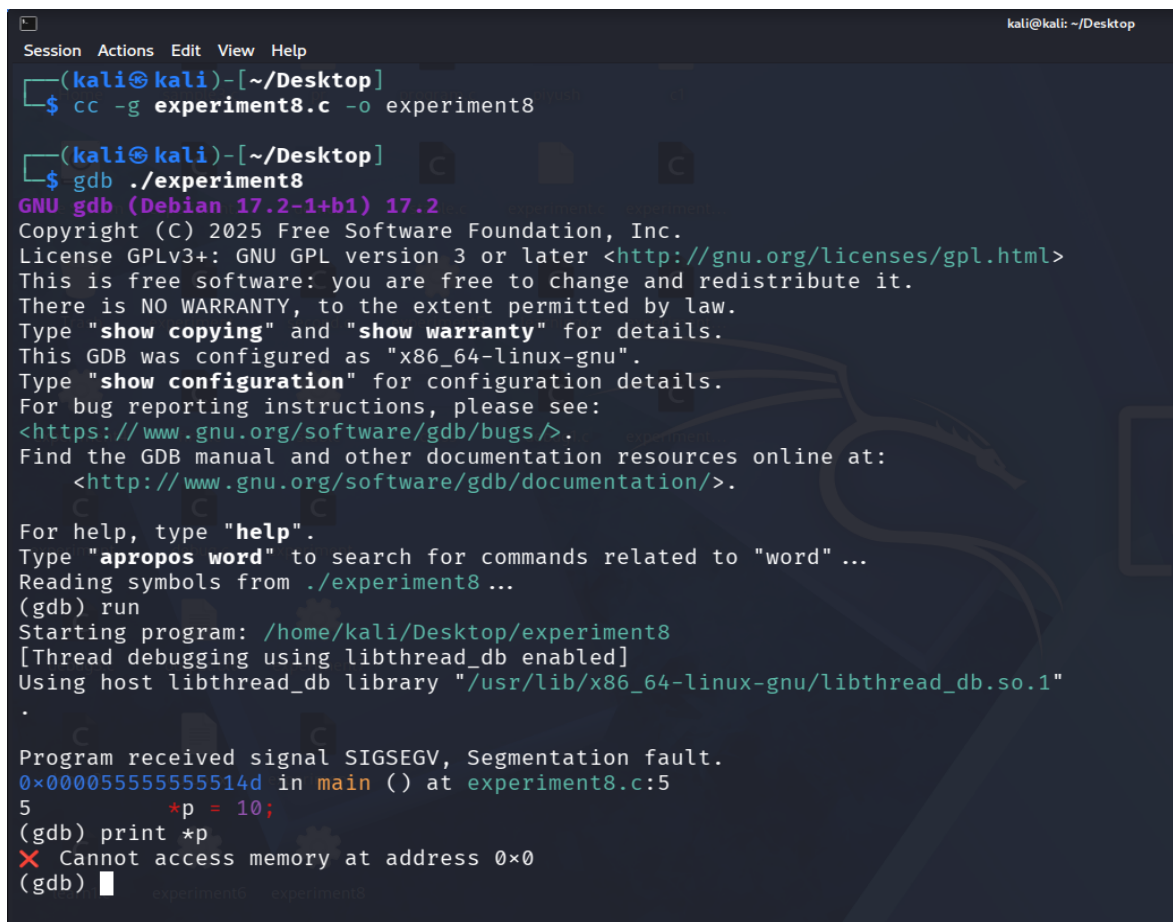
A Segmentation Fault occurs when a program attempts to access a memory location that it does not have permission to access (such as dereferencing a NULL pointer).

Example Code (experiment8.c)



```
1 #include<stdio.h>
2 int main()
3 {
4     int *p = NULL;
5     *p = 10;
6     printf("%d\n",p);
7     return 0;
8 }
```

Debugging in GDB



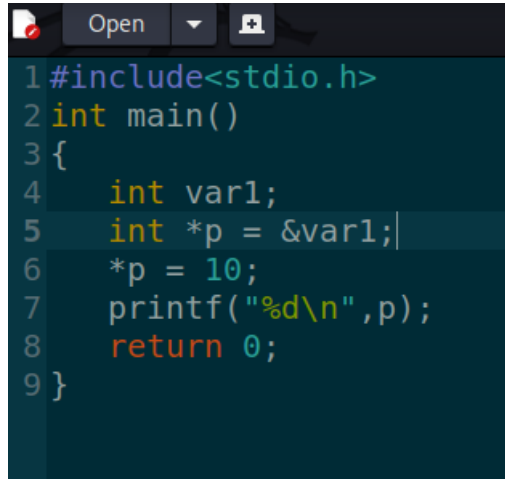
```
kali@kali: ~/Desktop
Session Actions Edit View Help
(kali@kali)-[~/Desktop]
$ cc -g experiment8.c -o experiment8

(kali@kali)-[~/Desktop]
$ gdb ./experiment8
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word" ...
Reading symbols from ./experiment8 ...
(gdb) run
Starting program: /home/kali/Desktop/experiment8
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
.

Program received signal SIGSEGV, Segmentation fault.
0x00005555555514d in main () at experiment8.c:5
5      *p = 10;
(gdb) print *p
✗ Cannot access memory at address 0x0
(gdb) █
```

Corrected Code

A screenshot of a code editor window with a dark theme. The window has a title bar with 'Open', a dropdown arrow, and a '+' icon. The code is written in C and is as follows:

```
1 #include<stdio.h>
2 int main()
3 {
4     int var1;
5     int *p = &var1;
6     *p = 10;
7     printf("%d\n",p);
8     return 0;
9 }
```

4. Systematic Debugging Strategy

To debug effectively, follow this 8-step methodology:

1. Observe Expected vs. Actual Output
2. Reproduce the Issue Consistently
3. Compile with Flags (-Wall -Wextra -g)
4. Isolate Suspicious Code Sections
5. Inspect State (printf or GDB breakpoints)
6. Apply a Single Minimal Fix
7. Recompile and Re-test
8. Verify with Edge Cases & Boundary Inputs

5. Exercises & Homework Questions

1. **Error Classification:** Write a small C program snippet illustrating each of the following error types: Syntax error, Compilation/Type error, Linker error, Runtime error, and Logical error.

- **Syntax Error**

```
#include <stdio.h>
int main()
{
    int x = 5 printf("Value: %d\n", x);
    return 0;
}
```

Error: expected ';' before 'printf'

- **Compilation / Type Error**

```
#include <stdio.h>
int main()
{
    int num = "Hello";
    printf("%d\n", num);
    return 0;
}
```

Error: warning/error: incompatible integer to pointer conversion or initialization of 'int' from incompatible pointer type

- **Linker Error**

```
#include <stdio.h>

int add(int a, int b);

int main()
{
    printf("%d\n", add(2, 3));
    return 0;
}
```

Error: undefined reference to 'add'.

- **Runtime Error**

```
#include <stdio.h>

int main()
{
    int a = 10, b = 0;
    printf("%d\n", a / b);
}
```

Error (at runtime): program crashes, often with Floating point exception (core dumped)

- **Logical Error**

```
int main()
{
    int a = 5, b = 10;
    int sum = a - b;
    printf("Sum: %d\n", sum);
    return 0;
}
```

2. GNU Concept: Explain what the GNU project is and how GDB fits into the GNU ecosystem.

GNU = a free software project (started 1983 by Richard Stallman) aiming to build a complete Unix-like OS made entirely of free/open tools. It gave us GCC (compiler), Bash (shell), Emacs (editor), and more. Linux (the kernel) combined with GNU tools to form GNU/Linux.

In short:

GCC → compiles your code.

Binutils → links it into an executable.

GDB → lets you debug it when something goes wrong.

3. Shell Context: Define what a Shell and Kernel are. Briefly list three common Linux shells (e.g., bash, zsh, fish).

Kerner: The kernel is the core of the operating system. The kernel works directly with the hardware, such as the processor the memory, the hard drive and other devices. The kernel decides how programs can use these parts of the computer. The kernel sits between the hardware and all the other things that are running on the system.

Shell: The shell is a program that lets people talk to the operating system. Usually this is done by typing commands. The shell listens to what you type understands what you want and then tells the kernel to do the work, such, as starting a program or organizing files. The shell sits above the kernel. Lets people use the system.

4. Research and contrast the differences between: Breakpoint, Watchpoint, Catchpoint, and Tracepoint.

Breakpoint

Pauses execution at a specific line, function, or instruction. The classic "stop here" tool. Good when you know *where* the bug might be.

Watchpoint

Pauses execution whenever a specific variable or memory address changes value — no matter which line causes it. Useful when you know *what* data is getting corrupted but not *where* it's happening.

Catchpoint

Pauses execution on a runtime event rather than a code location — things like an exception being thrown, a signal (e.g. SIGSEGV), or a system call. Useful for catching crashes or exception flow without knowing the exact line.

Tracepoint

Works like a breakpoint (tied to a code location), but instead of stopping the program, it logs data (variable values, timestamps, etc.) and lets execution continue. Useful for timing-sensitive bugs where pausing would change behavior.